

Database Transactions: An Introduction

A database transaction is a sequence of one or more operations, such as reads and writes, that a database system treats as a single unit of work. The defining property of a transaction is that it either completes in full or has no effect at all on the stored data.

Transactions exist because real applications perform multi-step updates that must stay coherent even when the system crashes, two users act at the same time, or a single step fails partway through. Without transactions, a crash midway through a multi-step update could leave the database in a state that no single valid operation would ever produce.

The guarantees that a transactional database system provides are commonly summarized by the acronym ACID: atomicity, consistency, isolation, and durability. The next four sections examine each guarantee individually, since students frequently confuse atomicity with durability even though they protect against different failures.

Atomicity

Atomicity means that a transaction is treated as a single indivisible unit: either every operation inside it is applied, or none of them are. There is no partially applied transaction visible to the rest of the system.

Consider a transaction that transfers money between two bank accounts by debiting one account and crediting another. If the debit succeeds but the system crashes before the credit is applied, atomicity requires that the database undo the debit when it recovers, rather than leaving the money missing from both accounts.

Atomicity is typically implemented using a write-ahead log combined with a commit or rollback mechanism. Before a transaction is allowed to commit, every operation it performed must be recorded in a way that lets the system either finish applying all of them or reverse all of them cleanly during recovery.

It is a common mistake to describe atomicity as guaranteeing that committed data survives a crash. That guarantee belongs to durability, described later in this lecture. Atomicity only guarantees that a transaction cannot be left half-finished; it says nothing on its own about what happens to data after a successful commit.

Consistency

Consistency means that a transaction moves the database from one valid state to another valid state, respecting all defined rules such as constraints, foreign keys, and triggers. A transaction that would violate a declared constraint is rejected rather than partially applied.

Consistency in the ACID sense is largely the responsibility of the application and schema design, not the transaction manager alone. The database enforces the constraints it has been told about, such as a column that must be unique or a foreign key that must reference an existing row, but it cannot enforce business rules that were never declared to it.

For example, a schema might declare that an account balance column can never be negative. A transaction that would produce a negative balance violates consistency and is rejected, protecting the database from ever storing a value the application considers invalid.

Isolation

Isolation means that concurrently executing transactions do not interfere with each other in ways that produce results different from some valid sequential ordering of those same transactions. Each transaction should behave as though it were the only transaction running, even when many are running at once.

Without isolation, one transaction could read data that another transaction has written but not yet committed, a problem known as a dirty read. If the writing transaction later rolls back, the reading transaction has already acted on data that never actually existed in the committed database.

Database systems offer several isolation levels, such as read committed, repeatable read, and serializable, each trading off strictness against performance. A stricter isolation level prevents more categories of anomaly but generally reduces how many transactions can proceed concurrently without blocking one another.

Durability

Durability means that once a transaction has been committed, its effects survive any subsequent crash, power loss, or restart of the database system. A committed transaction is never silently lost, even if the machine loses power one millisecond after confirming the commit to the application.

Durability is typically implemented by forcing the transaction log to persistent storage, such as a disk, before the system reports the commit as successful to the client. If the log entry is safely on persistent storage, the system can always replay it during crash recovery, even if the in-memory copy of the data was never written back before the crash.

This is precisely the property that distinguishes durability from atomicity. Atomicity protects a transaction while it is still in progress, ensuring it cannot be left half-done. Durability protects a transaction after it has already finished successfully, ensuring the completed result is never lost. A system could satisfy atomicity perfectly and still fail durability if it reported a commit as successful before safely persisting the log.

A practical way to keep the distinction straight: atomicity answers the question "did every step of this transaction happen, or none of them?" while durability answers the question "once the database told me it was done, can that answer ever change?"

Putting the Four Properties Together

A transactional database uses atomicity, consistency, isolation, and durability together to give application developers a simple mental model: a transaction either fully happens or fully does not happen, it never leaves the data in a state that violates the rules, it does not see or cause interference from other concurrent transactions, and once it succeeds, that success is permanent.

These four properties are independent guarantees. A system can be strong on some and weaker on others depending on its configuration, and understanding each guarantee separately is what allows an engineer to reason correctly about failure scenarios such as crashes during a multi-step update, concurrent access to the same row, or a machine losing power right after a commit is acknowledged.